MCSL-223

Computer Networks and Data Mining Lab

—

By Ritesh Tiwari

SECTION-A

Computer Networks Lab

1. Create a UDP client on a node n1 and a UDP server on a node n2.

Ans : Perfect 👍 Let's create a **UDP client–server setup** where:

- **Node n1** runs the **client** (sends a message).
- **Node n2** runs the **server** (receives and replies).

We'll implement this in **Python** because it's simple and works cross-platform.

🔹 **UDP Server (run on node n2)**

```
# udp_server.py
import socket
# Server IP & Port
SERVER_IP = "0.0.0.0"   # listen on all interfaces
PORT = 9999
# Create UDP socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_socket.bind((SERVER_IP, PORT))
print(f"UDP Server listening on port {PORT}...")
while True:
    # Receive message from client
    data, addr = server_socket.recvfrom(1024)
    print(f"Received from {addr}: {data.decode()}")
```

```
    # Send reply back to client
    reply = f"Hello Client, I got your message: {data.decode()}"
    server_socket.sendto(reply.encode(), addr)
```

- **UDP Client (run on node n1)**

```
# udp_client.py
import socket

# Replace with server's (n2) IP address
SERVER_IP = "192.168.1.2"   # Example, update with n2's actual IP
PORT = 9999

# Create UDP socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Send message
message = "Hello from Node n1"
client_socket.sendto(message.encode(), (SERVER_IP, PORT))

# Receive reply
data, _ = client_socket.recvfrom(1024)
print("Server says:", data.decode())

client_socket.close()
```

- **Steps to Run**

1. Save the files as **udp_server.py** and **udp_client.py**.

On **Node n2 (server machine)**:
python3 udp_server.py

2. (wait, it will listen on port 9999)

On **Node n1 (client machine)**:
python3 udp_client.py

3. Output example:

Server prints:
Received from ('192.168.1.1', 56789): Hello from Node n1

Client prints:
Server says: Hello Client, I got your message: Hello from Node n1

SECTION-B

Data Mining Lab

2. Demonstrate the preprocessing mechanism on a data set of your choice. Make necessary assumptions.

Ans :

I'll use the **Titanic dataset (simplified)** as an example, since it has both numeric and categorical values and also missing data.

🔹 **Assumptions**

- Dataset contains columns: `PassengerId, Name, Age, Gender, Fare, Survived`.
- Some values in **Age** are missing.
- **Gender** is categorical (Male/Female).
- **Fare** has outliers.
- Goal: Preprocess data for use in ML (e.g., classification).

🔹 **Preprocessing Steps**

1) Import Libraries & Sample Dataset

import pandas as pd

```
import numpy as np
# Sample dataset
data = {
    "PassengerId": [1, 2, 3, 4, 5],
    "Name": ["John", "Mary", "Sam", "Lucy", "Peter"],
    "Age": [22, np.nan, 35, 29, np.nan],
    "Gender": ["Male", "Female", "Male", "Female", "Male"],
    "Fare": [7.25, 71.83, 8.05, 512.32, 15.00],
    "Survived": [0, 1, 1, 1, 0]
}
df = pd.DataFrame(data)
print("Original Dataset:\n", df)
```

2) Handling Missing Values

- `Age` has missing values → fill them with **mean**.

df["Age"].fillna(df["Age"].mean(), inplace=True)

3) Encoding Categorical Values

- Convert `Gender` into numeric (Male=0, Female=1).

df["Gender"] = df["Gender"].map({"Male": 0, "Female": 1})

4) Outlier Handling

- `Fare` has extreme outlier (512.32).
- Use **log transformation** to reduce skew.

df["Fare"] = np.log1p(df["Fare"]) # log(1+Fare)

5) Normalization / Scaling

- Scale `Age` and `Fare` to [0,1] (Min-Max Scaling).

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[["Age", "Fare"]] = scaler.fit_transform(df[["Age", "Fare"]])
```

6) Final Preprocessed Dataset

```
print("Preprocessed Dataset:\n", df)
```

🔹 **Final Output (Example)**

Before Preprocessing

	PassengerId	Name	Age	Gender	Fare	Survived
0	1	John	22.0	Male	7.25	0
1	2	Mary	NaN	Female	71.83	1
2	3	Sam	35.0	Male	8.05	1
3	4	Lucy	29.0	Female	512.32	1
4	5	Peter	NaN	Male	15.00	0

After Preprocessing

	PassengerId	Name	Age	Gender	Fare	Survived
0	1	John	0.000000	0	0.008175	0
1	2	Mary	0.555556	1	0.254379	1
2	3	Sam	1.000000	0	0.009397	1
3	4	Lucy	0.666667	1	1.000000	1
4	5	Peter	0.555556	0	0.033081	0

✅ This shows **data preprocessing pipeline**:

- Handling missing values
- Encoding categorical variables
- Handling outliers
- Normalization

SECTION-A

Computer Networks Lab

1. Create a UDPClient and UDPServer nodes and communicate at a fixed data rate. Make necessary assumptions.

Ans :

- **Assumptions**

1. **Two nodes**:
   - Node n2 runs **UDPServer** (listens for packets).
   - Node n1 runs **UDPClient** (sends packets at fixed rate).
2. Network is reliable enough (UDP has no retransmission).
3. Data rate fixed in **packets per second (pps)**. For demo: **1 packet/second**.

- **UDP Server (Node n2)**

```
# udp_server.py
import socket
import time

# Server listens on this IP and port
SERVER_IP = "0.0.0.0"   # listen on all interfaces
PORT = 9999

# Create UDP socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_socket.bind((SERVER_IP, PORT))

print(f"[SERVER] Listening on port {PORT}...")

while True:
    data, addr = server_socket.recvfrom(1024)
    print(f"[{time.strftime('%H:%M:%S')}] Received from {addr}:
{data.decode()}")
```

🔹 **UDP Client (Node n1, fixed data rate)**

```
# udp_client.py
import socket
import time

# Server details (replace with server machine's IP)
SERVER_IP = "192.168.1.2"   # Example: n2 IP
PORT = 9999

# Fixed data rate: 1 packet per second
PACKETS_PER_SECOND = 1
INTERVAL = 1.0 / PACKETS_PER_SECOND  # seconds between packets

client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

print(f"[CLIENT] Sending packets to {SERVER_IP}:{PORT} at
{PACKETS_PER_SECOND} pkt/s")

count = 0
try:
    while True:
        count += 1
        message = f"Packet #{count} from client"
        client_socket.sendto(message.encode(), (SERVER_IP, PORT))
        print(f"[{time.strftime('%H:%M:%S')}] Sent: {message}")
        time.sleep(INTERVAL)   # wait before sending next
except KeyboardInterrupt:
    print("\n[CLIENT] Stopped sending.")
finally:
    client_socket.close()
```

🔹 **How to Run**

On **Node n2 (server)**:
python3 udp_server.py

1. On **Node n1 (client)**:
   python3 udp_client.py
2. Example Output:

Server:

[12:00:01] Received from ('192.168.1.1', 54022): Packet #1 from client

[12:00:02] Received from ('192.168.1.1', 54022): Packet #2 from client

Client:

[12:00:01] Sent: Packet #1 from client

[12:00:02] Sent: Packet #2 from client

👉 You can change `PACKETS_PER_SECOND` in **udp_client.py** to send at 10 pkt/s, 100 pkt/s, etc.

SECTION B

Data Mining Lab

2. Find the frequent patterns using FP-growth algorithm on any dataset(s) of your choice. Make necessary assumptions.

Ans :

🔹 **Assumptions**

- We have a **market basket dataset** (transactions in a store).
- Each transaction is a list of purchased items.
- Goal: Find **frequent itemsets** (sets of items often bought together).
- **Minimum support threshold** = 0.4 (40%).

🔹 **Sample Dataset**

T1: {Milk, Bread, Beer}

T2: {Milk, Bread}

T3: {Milk, Diaper, Beer, Eggs}

T4: {Bread, Diaper, Milk, Beer}

T5: {Bread, Diaper, Milk, Coke}

- **FP-Growth in Python**

We'll use **mlxtend library** (popular for pattern mining).

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth

# Step 1: Define dataset
dataset = [
    ["Milk", "Bread", "Beer"],
    ["Milk", "Bread"],
    ["Milk", "Diaper", "Beer", "Eggs"],
    ["Bread", "Diaper", "Milk", "Beer"],
    ["Bread", "Diaper", "Milk", "Coke"]
]

# Step 2: Encode transactions into one-hot format
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)

print("Transaction Dataset (One-hot Encoded):\n", df)

# Step 3: Apply FP-Growth
frequent_itemsets = fpgrowth(df, min_support=0.4, use_colnames=True)

print("\nFrequent Itemsets (support >= 0.4):")
print(frequent_itemsets)
```

- **Expected Output**

One-hot encoded dataset:

	Beer	Bread	Coke	Diaper	Eggs	Milk
0	True	True	False	False	False	True
1	False	True	False	False	False	True
2	True	False	False	True	True	True
3	True	True	False	True	False	True
4	False	True	True	True	False	True

Frequent itemsets (support ⩾ 0.4):

	support	itemsets
0	0.8	{Milk}
1	0.8	{Bread}
2	0.6	{Beer}
3	0.6	{Diaper}
4	0.8	{Milk, Bread}
5	0.6	{Milk, Diaper}
6	0.6	{Milk, Beer}
7	0.6	{Bread, Milk, Diaper}
8	0.4	{Bread, Milk, Beer}

🔹 **Interpretation**

- `{Milk}` and `{Bread}` appear in **80%** of transactions.
- `{Milk, Bread}` appears together in **80%** of transactions → strong pattern.
- `{Milk, Beer}` appears in **60%** of transactions.

- `{Bread, Milk, Diaper}` appears in **60%** of transactions → possible association rule like *"If Bread and Diaper, then Milk"*.

✅ We successfully applied **FP-Growth** to extract frequent patterns.

Would you like me to also extend this with **Association Rule Mining** (lift, confidence, conviction) so you can see actual *rules* generated from these frequent itemsets?

SECTION-A

Computer Networks Lab

1. Create a simple point to point network topology using two nodes. Make necessary assumptions.

Ans : Here's a minimal, working point-to-point topology with two nodes.

Topology (P2P)

[PC-A] <-- Cat5e/6 crossover (or auto-MDI/MDIX) --> [PC-B]

Assumptions

- Both nodes are PCs/NICs that support auto-MDI/MDIX (so a straight-through cable also works).
- No switch/router between them (true point-to-point).
- IPv4 used; no DHCP server, so we assign static IPs.
- Same link speed/duplex (Auto/Auto).

Addressing Plan (use a /30 for exactly two hosts)

Device	IP Address	Subnet Mask	Gateway
PC-A	192.168.10.1	255.255.255.252	—
PC-B	192.168.10.2	255.255.255.252	—

(Subnet 192.168.10.0/30 → usable hosts: .1 and .2, network .0, broadcast .3)

Setup Steps

1) Physical

- Connect PC-A NIC to PC-B NIC with Cat5e/6 cable (crossover preferred; straight-through OK if NICs support auto-MDI/MDIX).
- Verify link LEDs on both NICs.

2) Configure IPs

On Windows

- PC-A:
  Control Panel → Network & Internet → Change adapter settings → Right-click adapter → Properties → IPv4 → Use the following IP: `192.168.10.1`, mask `255.255.255.252`. Leave gateway blank.
- PC-B:
  Set IP `192.168.10.2`, mask `255.255.255.252`. Gateway blank.

(CLI alternative)

PC-A> netsh interface ip set address "Ethernet" static 192.168.10.1 255.255.255.252

PC-B> netsh interface ip set address "Ethernet" static 192.168.10.2 255.255.255.252

On Linux

PC-A

sudo ip addr add 192.168.10.1/30 dev eth0

sudo ip link set eth0 up

PC-B

sudo ip addr add 192.168.10.2/30 dev eth0

sudo ip link set eth0 up

3) Test Connectivity

From PC-A:

ping 192.168.10.2

From PC-B:

ping 192.168.10.1

You should see replies with low (<1 ms) latency.

Notes

- No default gateway needed since both hosts are on the same subnet and directly connected.
- If pings fail: check cable, disable firewalls temporarily, confirm both NICs are "up," ensure no IP conflicts, and verify the /30 mask on both ends.
- Optional IPv6 equivalent: `PC-A fe80::1/64`, `PC-B fe80::2/64` (or use link-local automatically), then `ping6` using interface scoping as needed.

This is the simplest possible point-to-point network: two nodes, a single cable, and /30 addressing.

SECTION B

Data Mining Lab

1. Implement simple k-means Algorithm to demonstrate the clustering rule on any dataset(s) of your choice.

Ans :

- **K-Means Algorithm (Steps)**

1. **Choose** `k` = number of clusters.
2. **Initialize centroids** randomly (pick `k` data points as starting centers).

3. **Assign step**: Assign each data point to the nearest centroid (Euclidean distance).
4. **Update step**: Compute new centroids as the mean of all points in each cluster.
5. **Repeat steps 3–4** until centroids don't change (or max iterations reached).
6. **Result**: Each point is labeled with its cluster.

🔹 **Python Implementation**

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# Step 1: Generate toy dataset
X, y_true = make_blobs(n_samples=200, centers=3, cluster_std=0.7,
random_state=42)

# Step 2: Implement K-Means
def kmeans(X, k, max_iters=100):
    # Randomly choose k points as initial centroids
    np.random.seed(42)
    centroids = X[np.random.choice(len(X), k, replace=False)]

    for _ in range(max_iters):
        # Assign: Compute distances & assign each point to nearest centroid
        distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
        labels = np.argmin(distances, axis=1)

        # Update: Compute new centroids
        new_centroids = np.array([X[labels == i].mean(axis=0) for i in
range(k)])

        # Stop if centroids do not change
        if np.all(centroids == new_centroids):
            break
        centroids = new_centroids

    return labels, centroids

# Step 3: Run K-Means
k = 3
labels, centroids = kmeans(X, k)
```

```
# Step 4: Visualize clusters
plt.scatter(X[:,0], X[:,1], c=labels, cmap='viridis', s=30)
plt.scatter(centroids[:,0], centroids[:,1], c='red', marker='X', s=200,
label="Centroids")
plt.title("K-Means Clustering Example")
plt.legend()
plt.show()
```

🔹 **Output (what you'll see)**

- Data points will be grouped into **3 clusters** with different colors.
- **Red "X" marks** indicate final cluster centroids.
- Each iteration adjusts cluster boundaries until convergence.

✅ This is a **from-scratch K-Means implementation** (no direct `sklearn.cluster.KMeans` used).

Do you want me to also show the **step-by-step centroid movement (iteration plots)** so you can visually see how clusters evolve, or just the final result is enough?